

✦ JOSA //

Code Review & the Maintainer Mindset

`/* Reviewing a stranger's code with the right balance of rigor and empathy */`

OpenLab Apprenticeship 2026

Week 04

// 1 //

Learning Goals

- Apply Google’s code review standard (“does this Change List (CL) improve overall code health?”) to real PRs.
- Review a stranger’s code constructively, with the right balance of rigor and empathy.
- Recognize over-engineering, premature abstraction, and unnecessary complexity.
- Begin thinking like a maintainer: gatekeeping with kindness.

// 2 //

Technical Deep Dive

// 2.1 The Standard (per Google)

Google’s eng-practices document is the most cited code review reference in the industry. The senior principle:



IMPORTANT

Reviewers should favor approving a Change List (CL) once it is in a state where it definitely improves the overall code health of the system being worked on, even if the CL isn’t perfect.

Read that twice. The bar isn’t “perfect.” The bar is **is the codebase better with this merged than without it?** If yes, and the disagreement is style or polish, approve and merge. Block only on real harm.

This sounds permissive but has a corollary: if the CL *doesn’t* improve code health (architectural problems, untested change, new abstraction with no caller, dead code, etc.), block it – and explain why.

// 2.2 What to actually look for?

A condensed checklist drawn from Google’s reviewer guide:

- **Design.** Does the change make sense? Does it fit the existing system or fight it?

- **Functionality.** Does it do what the description says? Does it handle edge cases (empty input, very large input, concurrency, errors)?
- **Complexity.** Is it more complex than it needs to be? Watch especially for over-engineering – generic solutions to problems we don’t have yet.
- **Tests.** Does the code have tests? Do they cover the new logic, not just happy paths?
- **Naming.** Are the names clear? Does the name `data` or `helper` appear? (If yes, push back.)
- **Comments.** Comments should explain *why*, not *what*. The code already says what.
- **Style.** Conform to the project style guide. Disagreements with the style guide go in a separate issue, not this CL.
- **Documentation.** Did public-facing changes update docs?

// 2.3 Comment style that works

Hard rules //////////////////////////////////

- **Comment on the code, not the developer.** Bad: “Why did you do this?” – Good: “This adds complexity that I can’t see a benefit for. Could we use the existing X helper instead?”
- **Tag severity.** Use Conventional Comments labels ((or anything similar that does the job). `nitpick (non-blocking)` tells the author this is a preference, not a merge-block.
- **Be specific.** “This needs error handling” is unactionable. “If parse throws on line 42, the file handle on line 38 leaks. Wrap in try/finally or use `with`.” is actionable.
- **Praise good work.** “Beautiful refactor – this is much clearer than what was there.” Code review is also mentorship; reinforcement matters.

// 2.4 Recognizing over-engineering

The most common failure mode in junior PRs is over-engineering. Signs:

- A new abstract class, interface, or trait with exactly one implementation.
- A new configuration option that no caller is going to use.
- A “future-proof” parameter that does nothing today.
- A function generalized to take a type when it’s only ever called with one type.
- A new module structure invented to “make things cleaner” with no clear failing pattern in the old structure.



TIP

The discipline: YAGNI – You Aint Gonna Need It. Build for what’s needed now. Generalize when you have at least two real callers, not before. When you see this in a CL, ask the author “what’s the second caller you have in mind?” – if there isn’t one, the abstraction is premature.

// 2.5 The maintainer's emotional load

Maintainers reject CLs all day. New contributors take rejection personally. This combination produces real burnout on the maintainer side. You can help by:

- Including a “Notes for reviewers” section that flags decisions the reviewer would otherwise have to ask about.
- Self-reviewing your own PR before requesting a review. Do a pass as if you were a stranger. You will catch 30% of issues yourself.
- Replying to feedback with “Thanks – fixed in {short-sha}” rather than long defenses. If you disagree, disagree once, clearly, and then defer.
- Saying thank you when a review takes time.

// 3 //////////////////////////////////

Learning Resources

Topic	Resource
The canonical reviewer guide	Google eng-practices: How To Do A Code Review
The CL author's perspective	Google eng-practices: CL Author's Guide
Comment labeling	Conventional Comments
Readable code (book)	The Art of Readable Code – Boswell & Foucher
Maintainer perspective (very niche but common opinion on why maintainers prefer mailing lists over modern tools like GitHub)	Greg Kroah-Hartman – Patches Carved Into Stone Tablets

// 4 //

Suggested Repositories

- **Any project where you've already submitted a PR.**
- [microsoft/vscode](#) – High volume of PRs; comments from Microsoft engineers are a masterclass.
- [rust-lang/rust](#) – Rust's review culture is documented and uniformly excellent. Reading is enough.
- [apache/superset](#) – Active Apache project with diverse contributor base.

// 5 //

Your Tasks for the Week

1. **Two real reviews.** Find three open PRs. Leave substantive comments on each, using Conventional Comments labels. Substantive means design or correctness, not formatting.
2. **Self-review write-up.** Take one of your own past PRs (the one you're least proud of). Pretend you're the reviewer. Write what you would have said. Write about it in your weekly journal.
3. **A "spot the over-engineering" exercise.** Find a real PR (open or merged) in any major project where you can identify either over-engineering or premature abstraction. Write in your weekly journal an analysis of: what's premature, what's the simpler alternative, and what would make the abstraction actually justified (if there was a new abstraction).
4. **Read some merged PRs in one project.** Pick a project with a strong review culture (e.g., Rust, Kubernetes, VS Code, Django). Read the discussion on multiple merged PRs. Write a summary of what their review norms look like – what gets flagged, what doesn't, how disagreements got resolved, etc..

Soft Skill of the Week: Giving and Receiving Feedback Without Ego

Code review is, structurally, a series of small criticisms. Both sides – reviewer and author – need techniques for not making it feel like an attack.

// 6.1 As the reviewer

- Lead with the code, not the person. (“This function” not “you wrote this.”)
- Tag severity. A [nitpick](#) is not a [blocker](#). Make this explicit.
- Sandwich is overrated. Don’t fake praise. Do offer real praise when you see real craft.
- Ask before stating. “Is there a reason X?” is often more accurate than “X is wrong” – sometimes there’s a reason you don’t know.

// 6.2 As the author

- Default to charity. If a comment feels harsh, assume the reviewer was rushed.
- Distinguish *I disagree* from *I don’t want to do the work*. The former is fine; defend it. The latter, fix.
- Say “you’re right, fixed” out loud – in writing, in the thread. It models the behavior you want from others.
- **Do not** rebase or force-push during review without saying so; reviewers lose context.



IMPORTANT

The deeper principle is **separating self from work**. Your code is not you. Even the best engineers ship bugs and write awkward abstractions. Code review is the system that catches them – not a verdict on your worth. Internalizing this is the difference between a senior and a junior, regardless of years of experience.

// 6.3 Resources

- [Crucial Conversations](#) – Patterson, Grenny, McMillan, Switzler. (The chapter on “making it safe” is directly applicable.)
- [Lara Hogan – Balancing direction and empowerment](#)